



BITS Pilani

Pilani | Dubai | Goa | Hyderabad

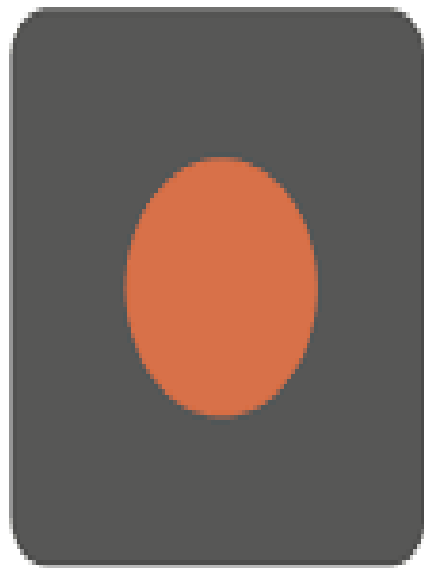
Compiler Design



Contact Session 7

LR(1) Parser Construction

IMP Note to Self



Start
Recording

IMP Note to Students

- It is important to know that just login to the session does not guarantee the attendance.
 - Once you join the session, continue till the end to consider you as present in the class.
 - IMPORTANTLY, you need to make the class more interactive by responding to Professors queries in the session.
 - **Whenever Professor calls your number / name ,you need to respond, otherwise it will be considered as ABSENT**
-

Objectives

- ✓ LR(1) Parser Construction
- ✓ CLR(1)
- ✓ LALR(1)
- ✓ Symbol Table
- ✓ Parser Generator

LR Parsers



- ✓ LR parsers present an efficient bottom up syntax analysis technique that can be used to parse a large class of context-free grammar.
- ✓ These techniques are called LR(k) parsing.
- ✓ L is for Left to right scanning of the input string.
- ✓ R is for constructing a rightmost derivation in reverse.
- ✓ K is for the number of input symbols of look ahead that are used in making parsing decision.
- ✓ When k is omitted, k is assumed to be 1.

Introduction Contd.

- ✓ LR parsing is attractive for the following reasons.
 - LR parsers can be constructed to recognize virtually all programming language constructs for which CFG can be written.
 - LR parsing method is the most general non-backtracking shift-reduce parsing method that can be implemented efficiently.
 - The LR parser can detect syntactic errors as soon as it is possible to do so on a left to right scan of the input string.
 - The class of grammars that can be parsed using LR methods is a proper superset of the class of grammars that can be parsed with predictive methods.

Introduction Contd.

- ✓ There are three common algorithms to build parsing tables for an “LR” parser:
- ✓ **Simple LR (SLR)**
 - smallest class of grammars and easiest to implement
 - smallest tables (number of states)
 - simple, fast construction
- ✓ **Canonical LR (CLR)**
 - full set of LR(1) grammars
 - largest tables (number of states)
 - slow, large construction
- ✓ **Look ahead LR (LALR)**
 - intermediate sized set of grammars
 - same number of states as SLR
 - canonical construction is slow and large
 - better construction techniques exist

Introduction Contd.

- ✓ CLR is more powerful than SLR and LALR
- ✓ LALR is more powerful than SLR
 - $\text{Power}(\text{CLR}) > \text{Power}(\text{LALR}) > \text{Power}(\text{SLR})$
- ✓ No. of states in CLR is more than SLR and LALR
- ✓ SLR and LALR has same number of states
 - $\text{States}(\text{CLR}) > \text{States}(\text{LALR}) \geq \text{States}(\text{SLR})$

LR(k) items

- ✓ The table construction algorithms use sets of LR(k) *items* or *configurations* to
- ✓ represent the possible states in a parse.
- ✓ An LR(k) item is a pair $[\alpha;\beta]$, where
 - α is a production from G with a ‘.’ (dot) at some position in the RHS marking how much of the RHS of a production has already been seen.
 - β is a look ahead string containing k symbols (terminals or \$).
- ✓ Two cases of interest are $k = 0$ and $k = 1$:
 - LR(0) items play a key role in the SLR table construction algorithm.
 - LR(1) items play a key role in the CLR and LALR table construction algorithms.

Canonical LR (CLR)- LR(1) items

- ✓ In the SLR method LR(0) items are considered.
- ✓ In CLR parsing LR(1) items are considered.
- ✓ LR (1) item is a collection of **LR (0) items and a look ahead symbol**
- ✓ **LR (1) item = LR (0) item + look ahead**
- ✓ The **look ahead** is used to determine that where we place the final item.
- ✓ The look ahead always add \$ symbol for the argument production.
- ✓ CLR refers to canonical lookahead.
- ✓ CLR parsing use the canonical collection of LR (1) items to build the CLR (1) parsing table.
- ✓ CLR (1) parsing table produces the more number of states as compare to the SLR (1) parsing.
- ✓ In the CLR (1), we place the reduce node only in the lookahead symbols.
- ✓ LR(1) parsers are more powerful parser. for LR(1) items we modify the Closure and GOTO function

Various steps involved in the CLR (1) Parsing:

- ✓ For the given input string write a context free grammar
- ✓ Check the ambiguity of the grammar
- ✓ Add Augment production in the given grammar
- ✓ Create Canonical collection of LR (0) items
- ✓ Draw a data flow diagram (DFA)
- ✓ Construct a CLR (1) parsing table

- ✓ The LR (1) item is defined by **production, position of data and a terminal symbol.**
- ✓ The terminal is called as Look ahead symbol.

General form of LR (1) item is

$$\begin{array}{c} S \rightarrow \alpha \bullet A \beta, \$ \\ A \rightarrow \bullet \gamma, \text{FIRST}(\beta, \$) \end{array}$$

Rules to create canonical collection:

1. Every element of I is added to closure of I .
2. If an LR (1) item $[X \rightarrow A \bullet BC, a]$ exists in I , and there exists a production $B \rightarrow b_1 b_2 \dots$, then add item $[B \rightarrow \bullet b_1 b_2, z]$ where z is a terminal in $\text{FIRST}(Ca)$, if it is not already in $\text{Closure}(I)$. keep applying this rule until there are no more elements to add.

EXAMPLE: The Grammar is given below, Construct the CLR(1) Table

$S \rightarrow CC$

$C \rightarrow cC$

$C \rightarrow d$

The Canonical collection of LR (1) items can be created as follows:



1. Augment Production for the Grammar

0. $S' \rightarrow \bullet S$

1. $S \rightarrow \bullet CC$

2. $C \rightarrow \bullet cC$

3. $C \rightarrow \bullet d$

2. Create Canonical collection of LR (0) items

I0 State : Add Augment production and compute the Closure, the look ahead symbol for the Augment Production is \$.

$$S' \rightarrow \bullet S, \$ = \text{Closure}(S' \rightarrow \bullet S, \$)$$

The dot symbol is followed by a Non terminal S. So, add productions starting with S in I0 State.

The dot symbol is followed by a Non terminal C. So, add productions starting with C in I0 State.

$S \rightarrow \bullet CC, \text{FIRST}(\$)$, using 2nd rule

$S \rightarrow \bullet CC, \$$

$C \rightarrow \bullet cC, \text{FIRST}(C, \$)$

$C \rightarrow \bullet d, \text{FIRST}(C, \$)$ $\text{FIRST}(C) = \{c, d\}$ so, the items are

$C \rightarrow \bullet cC, c/d$

$C \rightarrow \bullet d, c/d$.

The dot symbol is followed by a terminal value. So, close the I0 State. So, the productions in the **I0** are

$S' \rightarrow \bullet S, \$$

$S \rightarrow \bullet CC, \$$

$C \rightarrow \bullet cC, c/d$

$C \rightarrow \bullet d, c/d$

I1 = Goto (I0, S) = $S' \rightarrow S \bullet, \$$

I2 = Go to (I0, C) = Closure($S \rightarrow C \bullet C, \$$)

$S \rightarrow C \rightarrow \bullet cC, \$$

$C \rightarrow \bullet d, \$$ So, the I2 State is

$S \rightarrow C \bullet C, \$$

$C \rightarrow \bullet cC, \$$

$C \rightarrow \bullet d, \$$

I3 = Goto(I0, c) = Closure($C \rightarrow c \bullet C, c/d$)

$C \rightarrow \bullet cC, c/d$

$C \rightarrow \bullet d, c/d$

So, the I3 State is

$C \rightarrow c \bullet C, c/d$

$C \rightarrow \bullet cC, c/d$

$C \rightarrow \bullet d, c/d$

I4 = Goto(I0, d) = Closure(C- $\rightarrow$ d•, c/d) = C- $\rightarrow$ d•, c/d

I5 = Goto (I2, C) = closure(S- $\rightarrow$ CC•,\$)= S- $\rightarrow$ CC•, \$

I6 = Goto (I2, c) = closure(C- $\rightarrow$ c•C , \$)=C- $\rightarrow$ •cC, \$

C- $\rightarrow$ •d , \$ S0, the I6 State is

C- $\rightarrow$ c•C , \$

C- $\rightarrow$ •cC , \$

C- $\rightarrow$ •d,\$

I7 = Go to (I2 , d) = Closure(C- $\rightarrow$ d•,\$) = C- $\rightarrow$ d•, \$

Goto(I3, c) = closure(C- $\rightarrow$ •cC, c/d) = **I3**.

I8 = Go to (I3 , C) = Closure($C \rightarrow cC\bullet$, c/d) = $C \rightarrow cC\bullet$, c/d

Go to (I3 , c) = Closure($C \rightarrow c\bullet C$, c/d) = **I3**

Go to (I3 , d) = Closure($C \rightarrow d\bullet$, c/d) = **I4**

I9 = Go to (I6 , C) = Closure($C \rightarrow cC\bullet$, \$) = $C \rightarrow cC\bullet$, \$

Go to (I6 , c) = Closure($C \rightarrow c\bullet C$, \$) = **I6**

Go to (I6 , d) = Closure($C \rightarrow d\bullet$, \$) = **I7**

In Summary the States are :

$I_0 :$ $S \rightarrow \cdot S, \$$
 $S \rightarrow \cdot CC, \$$
 $C \rightarrow \cdot cC, c/d$
 $C \rightarrow \cdot d, c/d$

$I_1 :$ $S' \rightarrow S \cdot, \$$

$I_2 :$ $S \rightarrow C \cdot C, \$$
 $C \rightarrow \cdot cC, \$$
 $C \rightarrow \cdot d, \$$

$I_3 :$ $C \rightarrow c \cdot C, c/d$
 $C \rightarrow \cdot cC, c/d$
 $C \rightarrow \cdot d, c/d$

$I_4 :$ $C \rightarrow d \cdot, c/d$

$I_5 :$ $S \rightarrow CC \cdot, \$$

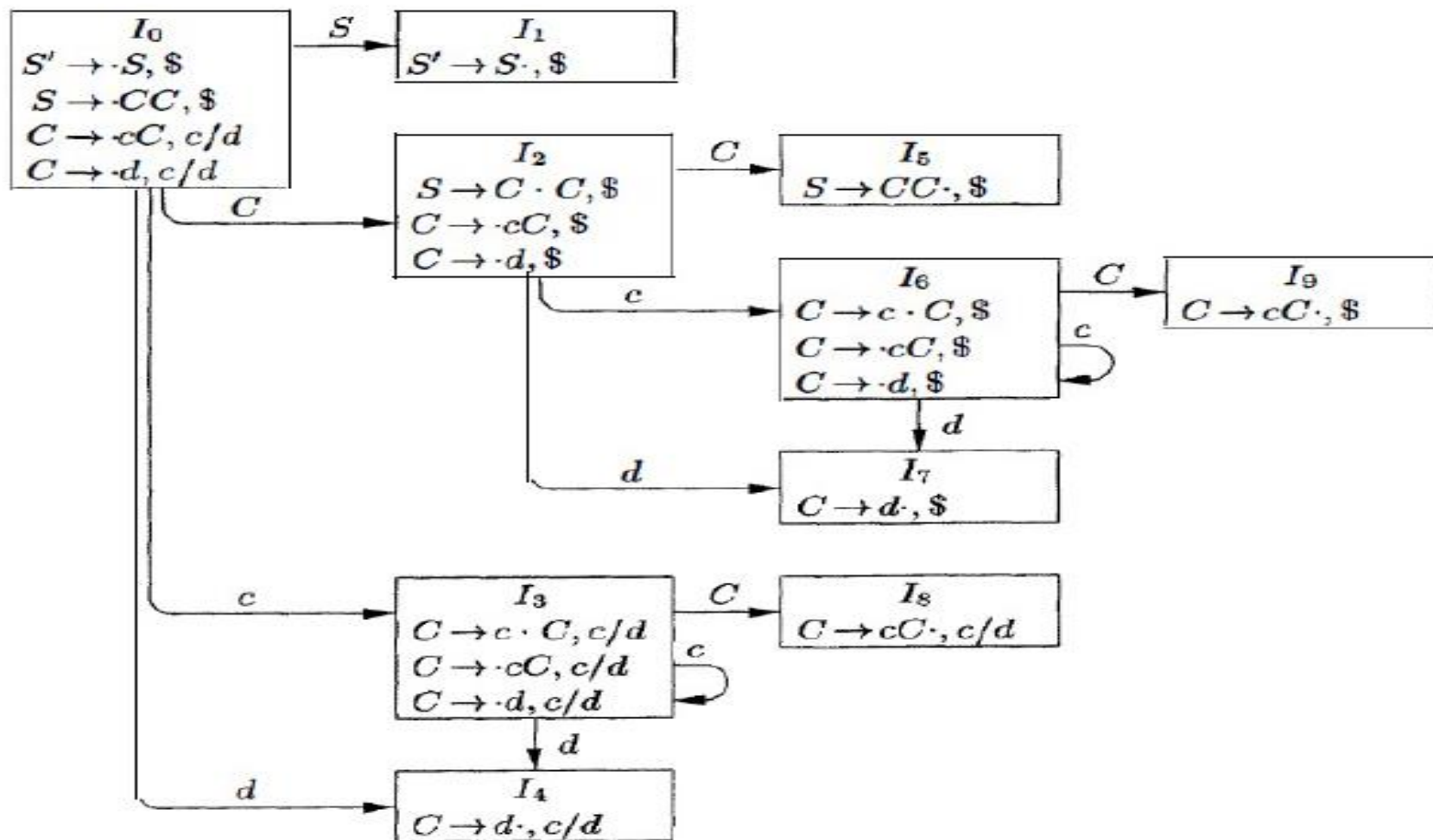
$I_6 :$ $C \rightarrow c \cdot C, \$$
 $C \rightarrow \cdot cC, \$$
 $C \rightarrow \cdot d, \$$

$I_7 :$ $C \rightarrow d \cdot, \$$

$I_8 :$ $C \rightarrow cC \cdot, c/d$

$I_9 :$ $C \rightarrow cC \cdot, \$$

DFA for Grammar



Construction of CLR (1) Table

- Rule1:** if there is an item $[A \rightarrow \alpha \bullet X \beta, b]$ in I_i and $\text{goto}(I_i, X)$ is in I_j then action $[I_i][X] = \text{Shift } j$, Where X is Terminal.
- Rule2:** if there is an item $[A \rightarrow \alpha \bullet, b]$ in I_i and $(A \neq S')$ set action $[I_i][b] = \text{reduce along with the production number}$.
- Rule3:** if there is an item $[S' \rightarrow S \bullet, \$]$ in I_i then set action $[I_i][\$] = \text{Accept}$.
- Rule4:** if there is an item $[A \rightarrow \alpha \bullet X \beta, b]$ in I_i and $\text{goto}(I_i, X)$ is in I_j then $\text{goto } [I_i][X] = j$, Where X is Non Terminal.

CLR(1) Parsing Table

States	ACTION			GOTO	
	C	D	\$	S	C
I0	S3	S4		1	2
I1			ACCEPT		
I2	S6	S7			5
I3	S3	S4			8
I4	R3	R3			5
I5			R1		
I6	S6	S7			9
I7			R3		
I8	R2	R2			
I9			R2		

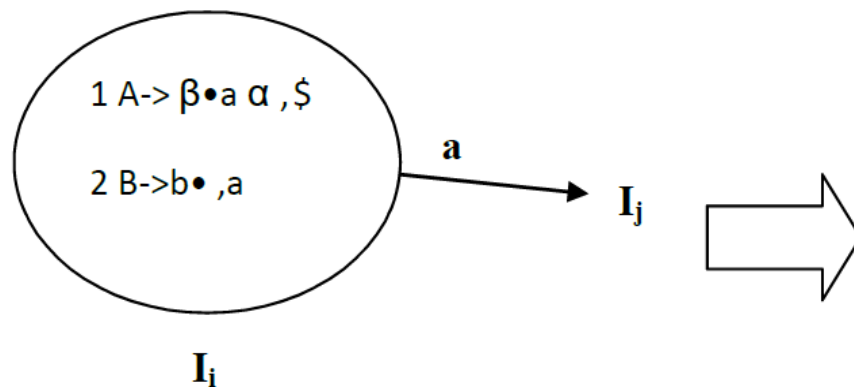
Conflicts in the CLR (1) Parsing

Conflicts in the CLR (1) Parsing : When multiple entries occur in the table. Then, the situation is said to be a Conflict.

Shift-Reduce Conflict in CLR (1) Parsing

Shift Reduce Conflict in the CLR (1) parsing occurs when a state has

3. A Reduced item of the form $A \rightarrow \alpha \bullet, a$ and
4. An incomplete item of the form $A \rightarrow \beta \bullet a \alpha$ as shown below:



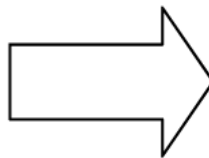
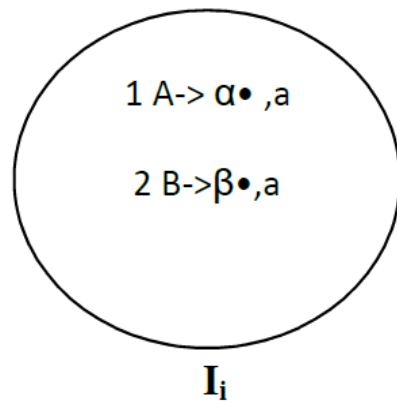
States	Action		GOTO	
	a	\$	A	B
I_i	Sj/r2			

Reduce / Reduce Conflict in CLR (1) Parsing

Reduce- Reduce Conflict in the CLR (1) parsing occurs when a state has two or more reduced items of the form

3. $A \rightarrow \alpha \bullet$

4. $B \rightarrow \beta \bullet$ If two productions in a state (I) reducing on same look ahead symbol as shown below:



States	Action		GOTO	
	a	\$	A	B
I_i	r_1/r_2			

String Acceptance using LR Parsing:

The input String is cdd

STACK	INPUT	ACTION
\$0	cdd\$	Shift S3
\$0c3	dd\$	Shift S4
\$0c3d4	d\$	Reduce with R3,C->d, pop 2*β symbols from the stack
\$0c3C	d\$	Goto (I3, C)=8Shift S6
\$0c3C8	d\$	Reduce with R2 ,C->cC, pop 2*β symbols from the stack
\$0C	d\$	Goto (I0, C)=2
\$0C2	d\$	Shift S7
\$0C2d7	\$	Reduce with R3,C->d, pop 2*β symbols from the stack
\$0C2C	\$	Goto (I2, C)=5
\$0C2C5	\$	Reduce with R1,S->CC, pop 2*β symbols from the stack
\$0S	\$	Goto (I0, S)=1
\$0S1	\$	Accept

LALR (1) Parsing(look ahead left right)

- ✓ The CLR Parser avoids the conflicts in the parse table.
- ✓ It produces more number of States when compared to SLR parser.
- ✓ More space is occupied by the table in the memory.

LALR Parsing

- ✓ Here, the tables obtained are smaller than CLR parse table.
- ✓ It is also as efficient as CLR parser.
- ✓ Here LR (1) items that have same productions but different look-aheads are combined to form a single set of items.

For example, consider the grammar in the previous example.

Consider the states I4 and I7 as given below:

$I4 = \text{Goto}(I0, d) = \text{Closure}(C \rightarrow d\bullet, c/d) = C \rightarrow d\bullet, c/d$

$I7 = \text{Go to}(I2, d) = \text{Closure}(C \rightarrow d\bullet, \$) = C \rightarrow d\bullet, \$$

These states are differing only in the look-aheads. They have the same productions.

Hence these states are combined to form a single state called as **I47**.

Similarly the states I3 and I6 differ only in their lookaheads as given below:

$I3 = \text{Goto}(I0, c) = C \rightarrow c\bullet C, c/d$

$C \rightarrow \bullet cC, c/d$

$C \rightarrow \bullet d, c/d$

$I6 = \text{Goto}(I2, c) = C \rightarrow c\bullet C, \$$

$C \rightarrow \bullet cC, \$$

$C \rightarrow \bullet d, \$$

These states are differing only in the lookaheads. They have the same productions.

Hence these states are combined to form a single state called as **I36**.

Similarly, the States I8 and I9 differ only in lookaheads. Hence they combined to form the state **I89**.

States	ACTION			GOTO	
	c	d	\$	S	C
I ₀	S ₃₆	S ₄₇		1	2
I ₁			ACCEPT		
I ₂	S ₃₆	S ₄₇			5
I ₃₆	S ₃₆	S ₄₇			89
I ₄₇	R ₃	R ₃	R ₃		5
I ₅			R ₁		
I ₈₉	R ₂	R ₂	R ₂		

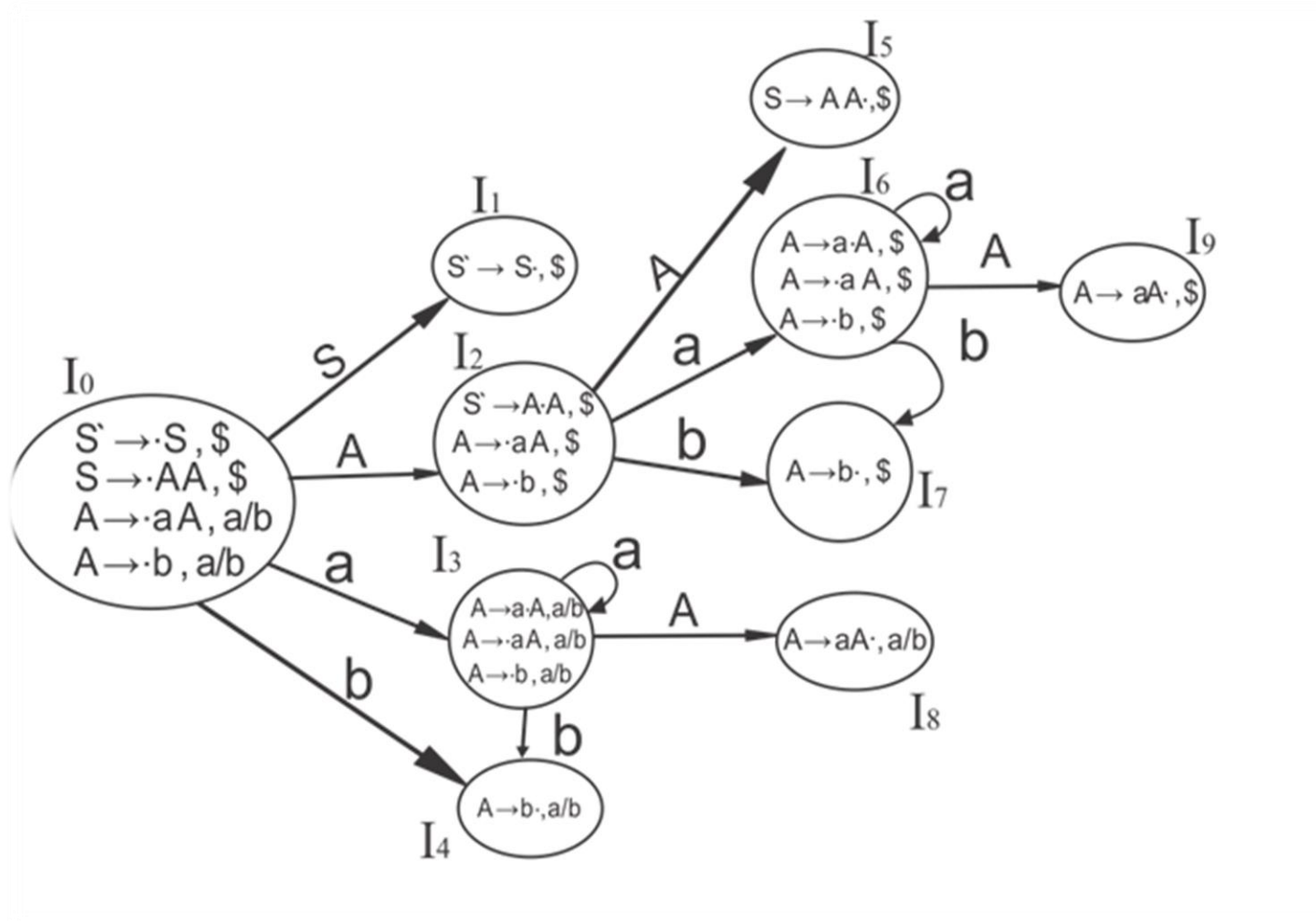
Example-2



CLR (1) Grammar

1. $S \rightarrow AA$
2. $A \rightarrow aA$
3. $A \rightarrow b$

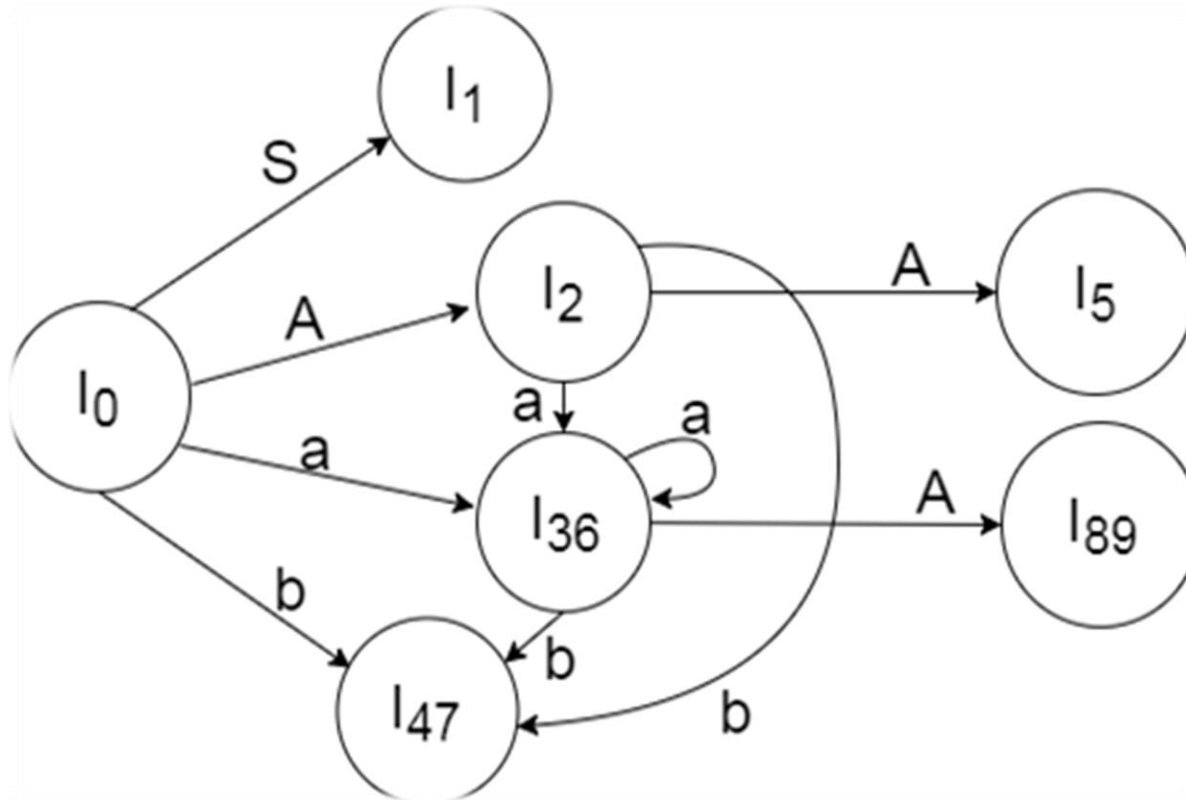
DFA for CLR Parsing



CLR(1) Parsing Table

States	a	b	\$	S	A
I ₀	S ₃	S ₄			2
I ₁			Accept		
I ₂	S ₆	S ₇			5
I ₃	S ₃	S ₄			8
I ₄	R ₃	R ₃			
I ₅			R ₁		
I ₆	S ₆	S ₇			9
I ₇			R ₃		
I ₈	R ₂	R ₂			
I ₉			R ₂		

DFA for LALR PARSER-Example 2



LALR Parsing Table

States	a	b	S	S	A
I ₀	S ₃₆	S ₄₇		12	
I ₁		accept			
I ₂	S ₃₆	S ₄₇			5
I ₃₆	S ₃₆ S ₄₇				89
I ₄₇	R ₃ R ₃	R ₃			
I ₅			R ₁		
I ₈₉	R ₂	<u>R₂</u>	<u>R₂</u>		

Difference between SLR, CLR, LALR

SLR Parsers	Canonical LR Parsers	LALR Parsers
SLR parser are easiest to implement.	CLR parsers are difficult to implement.	LALR parsers are difficult to implement than SLR parser but less than CLR parsers.
SLR parsers make use of canonical collection of LR(0) items for constructing the parsing tables.	CLR parsers are uses LR(1) collection of items for constructing the parsing tables part.	LALR parsers LR(1) collection , items with items having same core merged into a single itemset.
SLR parsers don't do any lookahead i.e., they lookahead zero	CLR parsers lookahead one symbol.	LALR parsers lookahead one symbol.
SLR parsers are cost effective to construct in terms of time and space.	CLR parsers are expensive to construct in terms – of time and space.	The cost of constructing LALR parsers is i intermediate between SLR and CLR parser.
SLR parsers uses FOLLOW information to guide reductions.	CLR parsers uses lookahead symbol to guide reductions.	LALR parsers uses lookahead symbol to guide reductions.

SLR Parsers	Canonical LR Parsers	LALR Parsers
SLR parsers may fail to produce a table for certain class of grammars on which they succeed.	CLR parser works on very large class of grammar.	LALR parser works on very large class grammars.
Every SLR(1) grammar is LR(1) grammar and LALR(1).	Every LR(1) grammar may not be SLR(1) grammar.	Every LALR(1) grammar may not be SLR(1) but every LALR(1) grammar is LR(1) grammar.
A shift-reduce or reduce-reduce conflict may arise in SLR parsing table.	A shift-reduce or reduce-reduce conflict may arise but chances are less than that in SLR parsing tables.	A shift-reduce conflict can not arise but a reduce-reduce conflict may arise.
SLR parser is least powerful.	A CLR parser is most powerful among the family of canonical LR bottom-up parsers.	A LALR parser is intermediate in power between SLR and LR parser.

Symbol Tables

The Symbol Table

The symbol table is used to store essential information about every symbol contained within the program.

This includes:

- ✓ Keywords
 - ✓ Data Types
 - ✓ Operators
 - ✓ Functions
 - ✓ Variables
 - ✓ Procedures
 - ✓ Constants
 - ✓ Literals
-

Contents of the Symbol Table

innovate

achieve

lead

The symbol table will contain the following types of information for the input strings in a source program:

- ✓ The lexeme (*input string*) itself
- ✓ Corresponding token
- ✓ Its semantic component (e.g., variable, operator, constant, functions, procedure, etc.)
- ✓ Data type
- ✓ Pointers to other entries (when necessary)

The interaction between the Symbol Table and the phases of a compiler



Virtually every phase of the compiler will use the symbol Table

- ✓ The initialization phase will place keywords, operators, and standard identifiers in it.
 - ✓ The scanner will place user-defined identifiers and literals in it and will return the corresponding token.
 - ✓ The parser uses these token to create the parse tree, the product of the syntactic analysis of the program.
 - ✓ The semantic action routines place data type in its entries and uses this information in performing basic type checking.
- 

The interaction between the Symbol Table and the phases of a compiler (continued)

innovate

achieve

lead

- ✓ The intermediate code generation phase use pointers to entries in the symbol table in creating the intermediate representation of the program
- ✓ The object code generation phase uses pointers to entries in the symbol table in allocating storage for its variables and constants, as well as to store the addresses of its procedures and functions.

Looking things up in the Symbol Table

Example - How would the scanner read a Pascal program header:

int main(void)

After reading the first 4 characters, it assembles the lexeme.

'i'	'n'	't'	"
-----	-----	-----	---

Pushing back the blank into the input stream. Since **int** is a reserved word, it is already in the symbol table, which returns the token **int**

Looking things up in the Symbol Table

Next, the scanner assembles the lexeme pushing back the (following it

'm'	'a'	'i'	'n'
-----	-----	-----	-----

The Symbol Table manager can't find it when it looks it up. Therefore, it installs the **main** in the Symbol Table with the token **identifier**.

During semantic analysis, it will be given the semantic property of **int**.

Data stored in the Symbol Table



Let's take a look at the kinds of information stored in the symbol table for different types of elements within a program.

	Token	Symbol	Data		
<u>Lexeme</u>	<u>Class</u>	<u>Type</u>	<u>Type</u>	<u>Value</u>	<u>Scope</u>
while	TokWhile	Keyword	None	0	Global
+	TokPlus	Operator	None	0	Global
x	TokIdentifier	Variable	Integer	0	Sample
8	TokConstant	Literal	Integer	8	Global



The basic operations of the Symbol Table

There are several operations that are performed on the Symbol Table, the most important being:

- **Adding symbols** - The reserved words, standard identifiers and operators are placed in the Symbol Table during its initialization.
 - New lexemes are added when the scanner encounters them, and they are assigned a token class.
 - Similarly, the semantic analyzer adds the appropriate properties and attributes that belong to the lexeme.
-

Deleting symbols - When the compiler is finished with a given scope of the program, all the symbols belonging to that scope must be effectively removed before beginning to process a different scope of the program.

- The data regarding these variables may be hidden from the compiler's view or dumped into a temporary file.
- **Searching for symbols** - Looking up a lexeme in the symbol table and retrieving any and all of the properties belonging to that symbol

Possible organization methods

There are several different ways to organize the symbol table:

The most obvious is to organize it as an ***array of records***, but it would either require a linear search or constant sorting.

begin	tokbegin	stkeyword	dtnone	...	global
call	tokcall	stkeyword	dtnone	...	global
called	tokidentifier	stvariable	dtreal	...	sample
procedure	tokprocedure	stkeyword	dtnone	...	global
xmin	tokidentifier	tokconstant	dtinteger	...	sample

We could use an ***unordered list***. This would make efficient use of memory and it would be quick and easy to install names, but retrieval would be unacceptably slow.

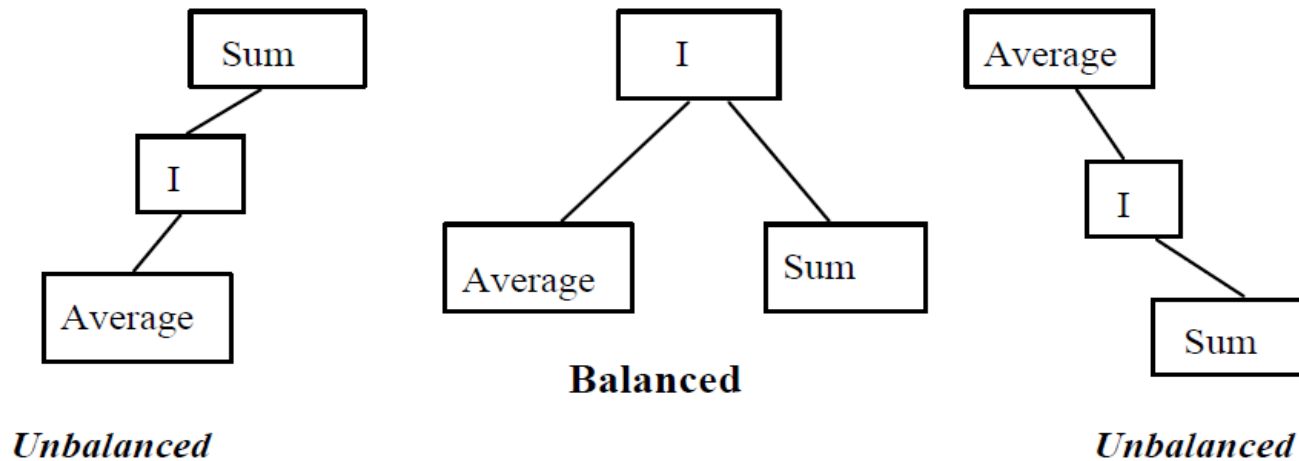


An **ordered list** would make storing and retrieving names faster, but not by that much.



Binary search trees would provide some compromises on these structures above, but only when the tree is balanced.

This rarely happens on its own and takes time for balancing to be done by the insertion routine.



The Attribute table

- The attribute table can be considered to be the central structure of the symbol table where the attributes of each lexeme are stored.
- The attribute table is an array of records in which we store data relevant to the symbol:
 - Symbol Type
 - Data Type
 - Token class
 - Value
 - Pointer back to the Name Table

The data in the attribute table is separated from the name table because the same name can be used in different contexts in different scopes of the program.

For this reason, it is best to use a pointer to the attribute table entry directly in most cases.

- If a name that already is used in outer scope is used again, we will create another attribute table entry and link it to the outer scope's entry.

Scope rules



- Block-structured languages, such as ALGOL, PL/I, Pascal and C, have to allow for the fact that the same identifier may mean completely different things in different scopes.
- Strictly speaking, a block is a program structure in which there can be declarations. E.g., Although it is unusual to place declarations in C blocks (excluding the main block of a function), it is perfectly legal.
- Because such languages resolve name conflicts using a most closely nested scope rule, languages with static scoping rules must have a mechanism for temporarily hiding a variable in an outer scope.
- In implementing a symbol table, this mean either:
 - Separate symbol tables for each scope or
 - identifying the scope to which each identifier belongs.

Opening and closing the scope

Opening the scope involves:

- Creating a new entry in the attribute table.
- Readjusting the name table pointer to point to the new entry (and vice versa).
- Linking the new entry to the other entries in the new scope as well as to the other entries sharing its name.

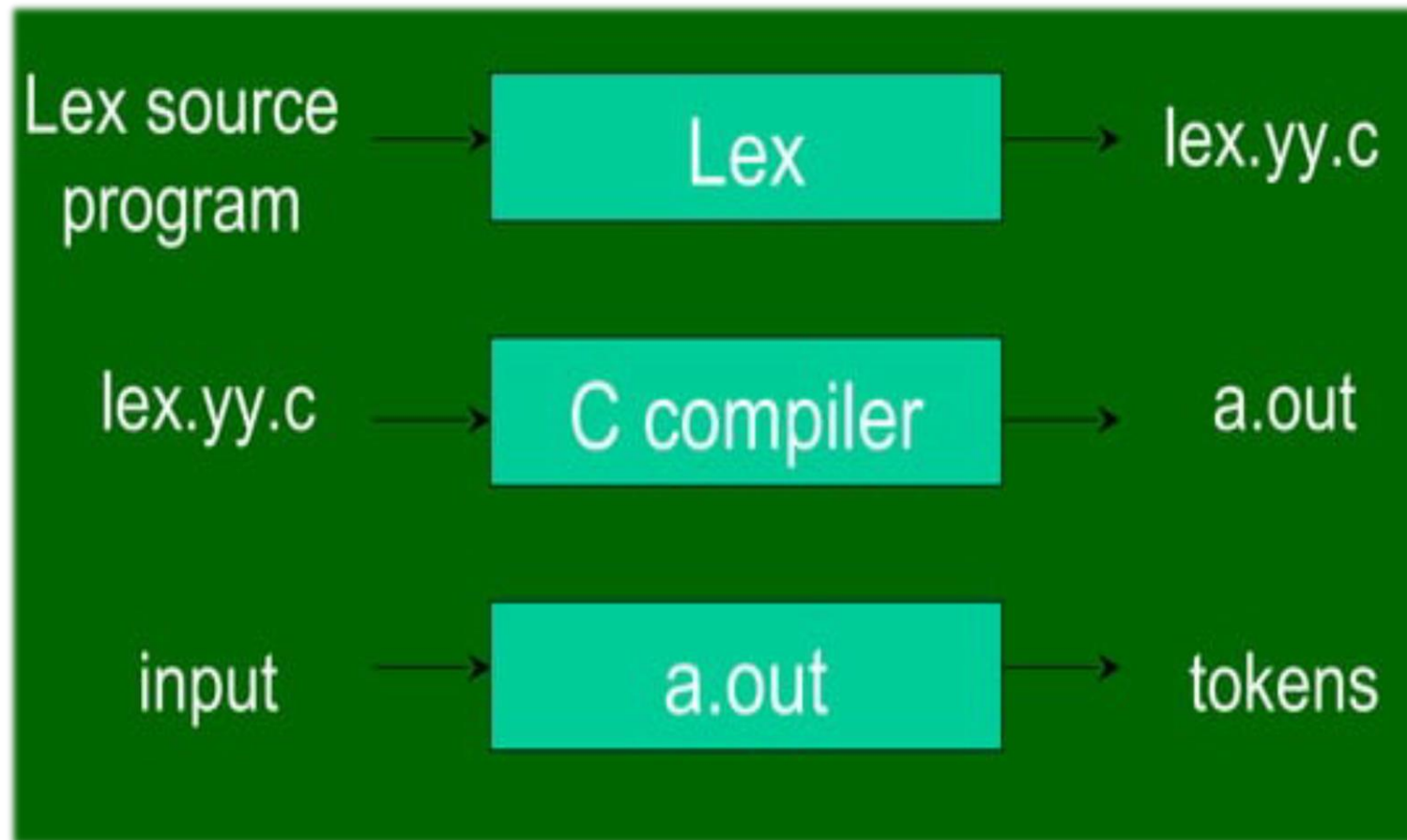
Closing the scope involves:

- Readjusting the pointer in the name table to point to the next scope outwards for that identifier and possibly,
 - writing the data on disk and clearing the entry.
-

Parser Generator-YACC

- ✓ Yacc – **YET ANOTHER COMPILER COMPILER.**
- ✓ An LALR(1) parser generator.
- ✓ Yacc generates
 - ✓ Tables – according to the grammar rules.
 - ✓ Driver routines – in C programming language.
 - ✓ y.output – a report file.

An Overview of the LEX



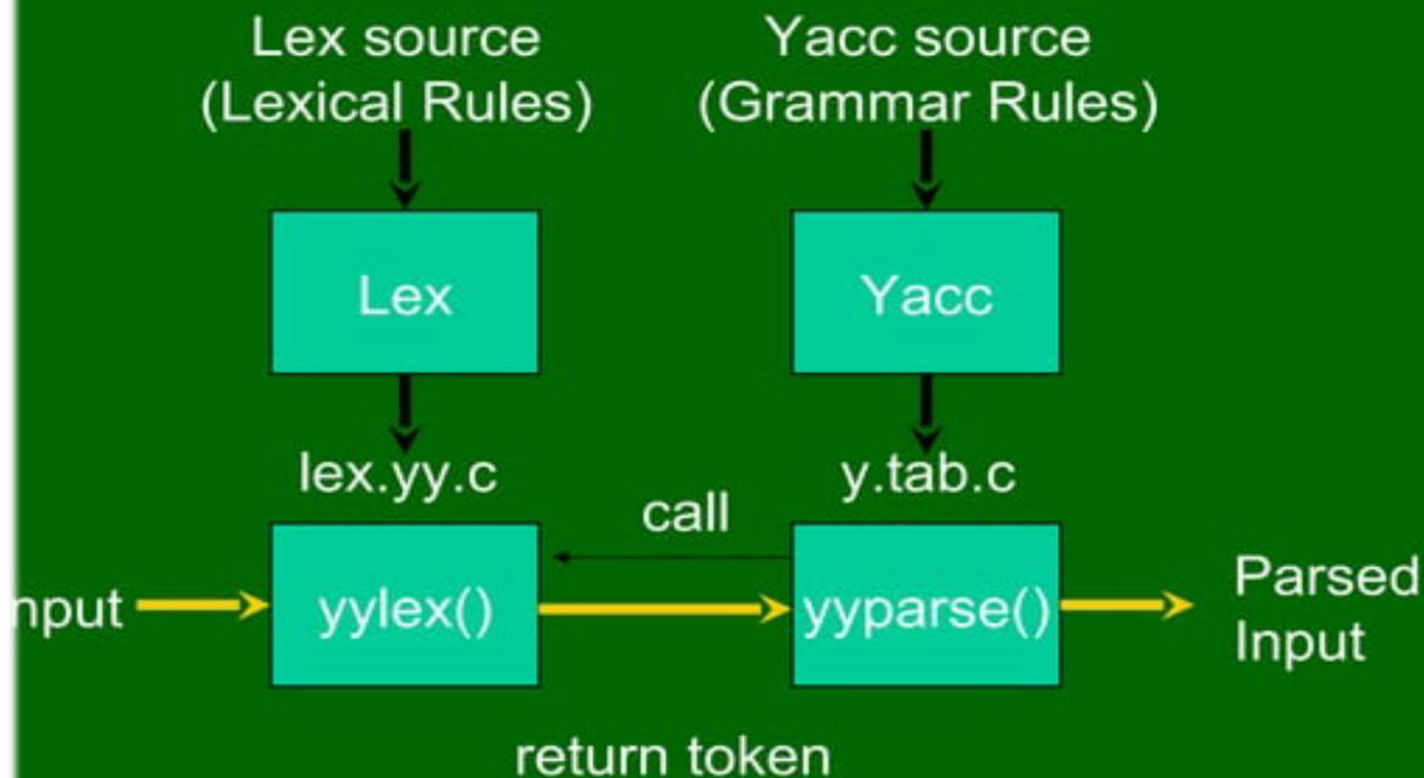
Lex

- Lex generates C code for a lexical analyzer, or **scanner**
- Lex uses patterns that match strings in the input and converts the strings to tokens

Yacc

- Yacc generates C code for syntax analyzer, or **parser**.
- Yacc uses grammar rules that allow it to analyze tokens from Lex and create a syntax tree.

Lex with Yacc



Lex-predefined variables.

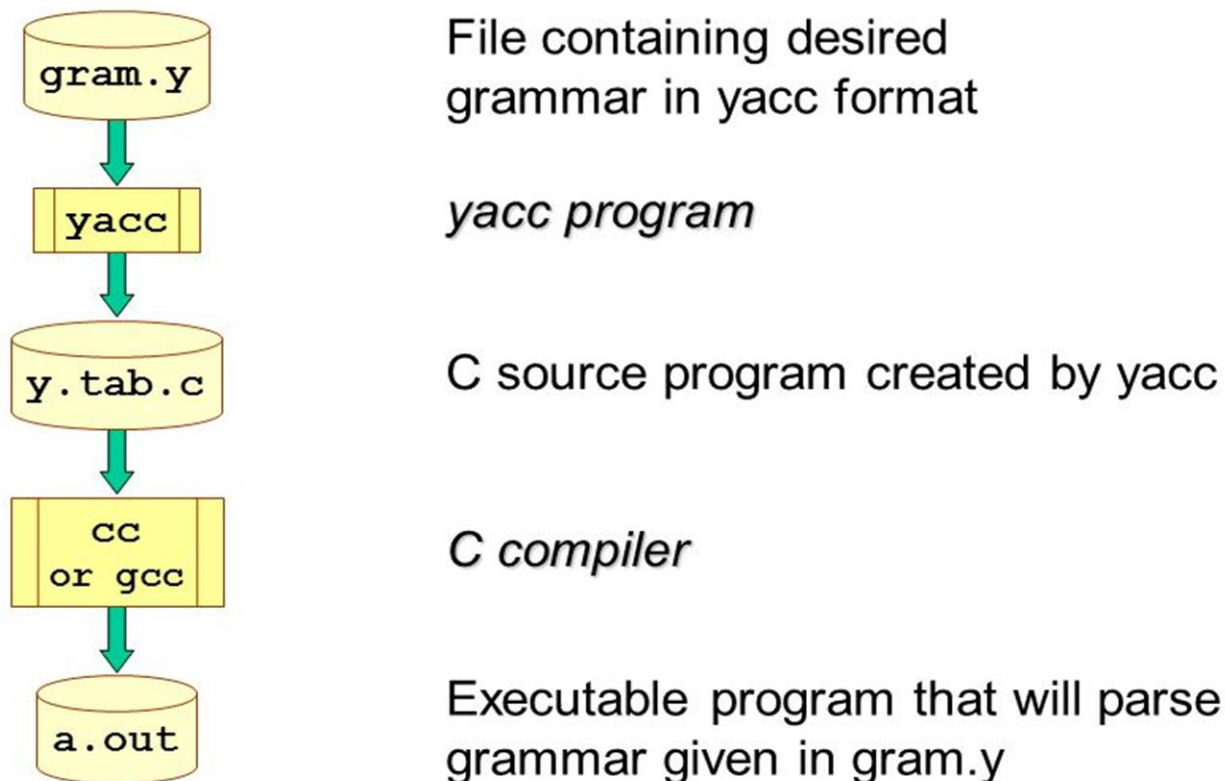


Name	Function
<code>int yylex(void)</code>	call to invoke lexer, returns token
<code>char *yytext</code>	pointer to matched string
<code>yylen</code>	length of matched string
<code>yyval</code>	value associated with token
<code>int yywrap(void)</code>	wrapup, return 1 if done, 0 if not done
<code>FILE *yyout</code>	output file
<code>FILE *yyin</code>	input file
<code>INITIAL</code>	initial start condition
<code>BEGIN</code>	condition switch start condition
<code>ECHO</code>	write matched string

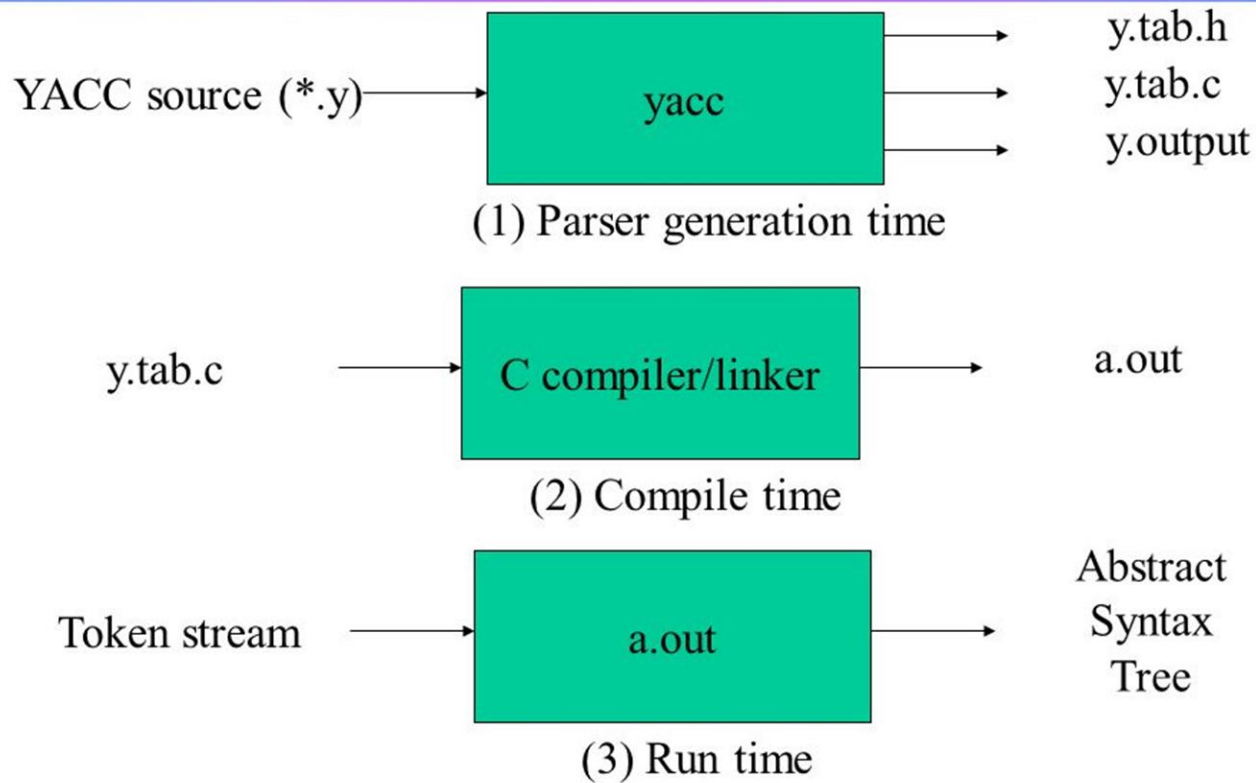
Parser Generator -YACC

- ✓ Invokes `yylex()` automatically.
- ✓ Generate `y.tab.h` file through the `-d` option.
- ✓ The lex input file must contains `y.tab.h`
- ✓ For each token that lex recognized, a number is returned (from `yylex()` function.)

How YACC Works



How YACC Works



- ✓ Yacc reads the grammar and generate C code for a parser .
- ✓ Grammars written in Backus Naur Form (BNF) .
- ✓ BNF grammar used to express *context-free languages* .
- ✓ e.g. to parse an expression , do reverse operation(reducing the expression)
- ✓ This known as *bottom-up or shift-reduce parsing* .
- ✓ *Using* stack for storing (LIFO).

Structure of the YACC

Input to yacc is divided into three sections.

... definitions ...

%%

... rules ...

%%

... subroutines ...

- ✓ **The definitions section consists of:**
 - ✓ token declarations.
 - ✓ C code bracketed by “%{“ and “%}”.
- ✓ **The rules section consists of:**
BNF grammar.
- ✓ **The subroutines section consists of:**
user subroutines.

The grammar:

$\text{program} \rightarrow \text{program expr} \mid \epsilon$

$\text{expr} \rightarrow \text{expr} + \text{expr} \mid \text{expr} - \text{expr} \mid \text{id}$

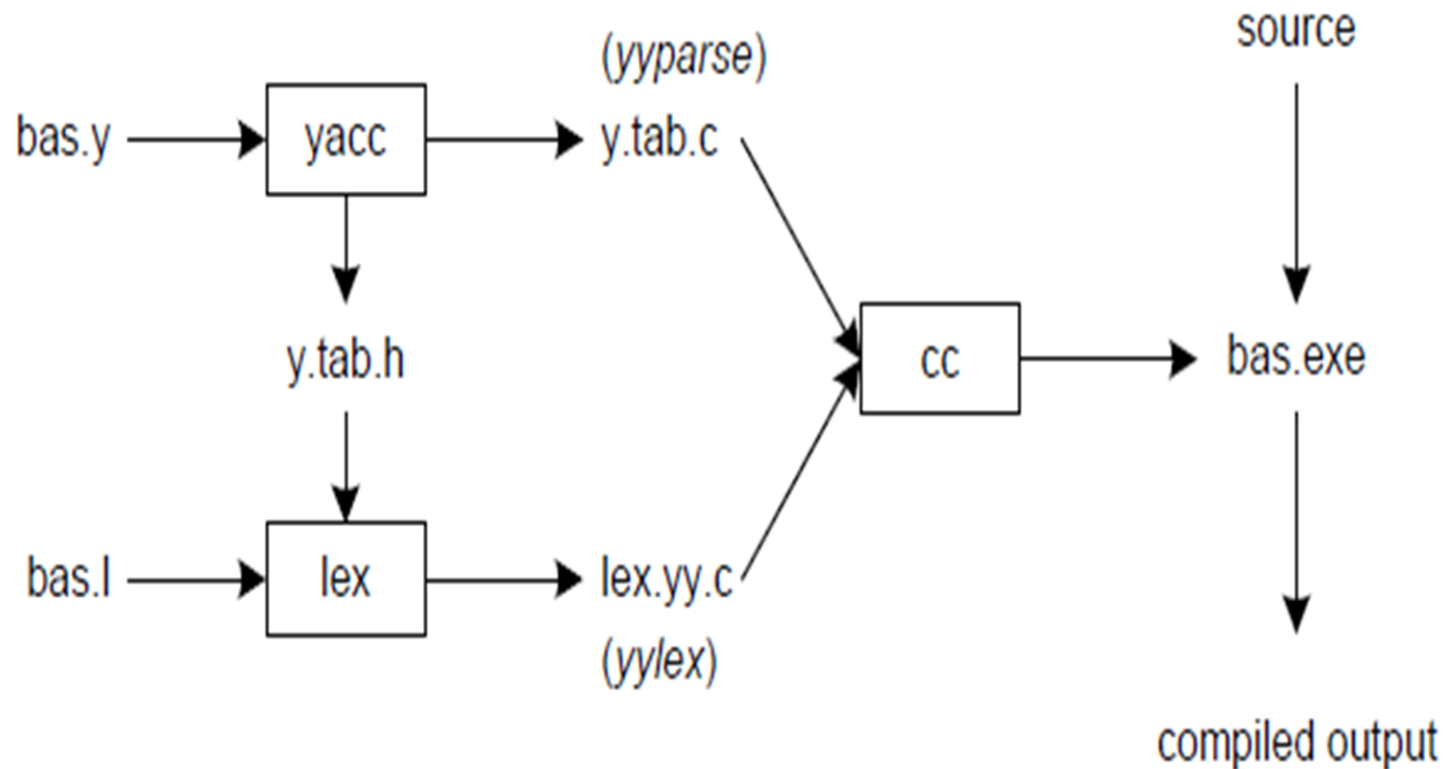
Program and expr are nonterminals.

Id are terminals (tokens returned by lex) .

expression may be :

- sum of two expressions .
- product of two expressions .
- Or an identifiers

Linking lex&yacc



IMP Note to Self



References

<https://www.javatpoint.com/clr-1-parsing>

<https://www.jntua.ac.in/gate-online-classes/registration/downloads/material/a159280488654.pdf>

<https://www.csie.ntu.edu.tw/~b93501005/slide5.pdf>

Thank you